PROJECT VT1 · MANAGEMENT REVIEW

Observability for AI Agents

Open-source tooling for tracing, evaluating, and governing agentic systems

Author: Daniela Otelea

Supervisor: Dr. Leonardo Militano

Programme: ZHAW · VT1 · Semester 3

Submission date: 30 June 2026

1 Executive Summary

This project investigated how to make AI agents observable and debuggable using open-source tooling, and delivered a head-to-head evaluation of three platforms — **Arize Phoenix**, **Langfuse**, and **Comet Opik** — across progressively more complex agent architectures.

Three working systems were built and instrumented end-to-end: a single *ReAct* agent, a multi-agent research-and-fact-check pipeline running in-process, and a distributed variant of that pipeline using Google's Agent2Agent (A2A) protocol. Each system was traced through all three platforms over a controlled experiment set, five query workloads per exporter, producing directly comparable latency, cost, and trace-quality data across 15 instrumented runs.

> Single-agent tracing is effectively a solved problem: all three tools capture reasoning steps, tool calls, and token cost with minimal integration effort and near-identical results. The real differentiation appears in multi-agent and especially distributed settings, where cross-process session correlation, semantic-convention consistency, and agent-specific quality/governance signals separate the platforms.

2 Context & Motivation

AI agents are moving from demos into production, where they plan, call tools, and coordinate with other agents autonomously. That autonomy is exactly what makes them hard to operate: a single user request can fan out into dozens of LLM calls, tool invocations, and inter-agent

messages — any of which can fail silently, hallucinate, loop, or leak sensitive data.

Observability addresses the operational risks specific to agentic systems:

- **Debugging** — reconstructing *why* an agent produced an answer requires the full reasoning trace, not just inputs and outputs.
- **Cost** — token usage compounds across reasoning loops and agent roles; without per-role attribution, spend stays invisible until the bill arrives.
- **Quality** — hallucination and incompleteness are silent failures that conventional monitoring never surfaces.
- **Safety & governance** — prompt injection, PII leakage, and runaway loops must be detected and audited.
- **Distributed coordination** — once agents run as independent services, failures hide in the gaps between them.

Classic application performance monitoring was built for deterministic request/response services. It has no native concept of a reasoning trace, a tool-call span, an inter-agent message, or a hallucination score — precisely the signals agent operators need.

3 Objectives & Scope

Objectives

- Define what must be observed in an AI agent — a structured requirements framework.
- Survey the open-source landscape and select a representative subset for in-depth evaluation.
- Build instrumented reference agents and evaluate the selected tools against the framework.
- Derive practical guidance and a tool recommendation.

Scope

- **In scope:** three open-source platforms (Arize Phoenix, Langfuse, Comet Opik); OpenTelemetry-based instrumentation; single-agent, multi-agent in-process, and distributed (A2A) architectures; latency, cost, trace quality, session correlation, and governance signals.
- **Out of scope:** production-scale load testing; proprietary/commercial platforms; multi-turn conversational workloads; security penetration testing.

The evaluation runs in two rounds plus a distributed extension: **Round 1** (single agent) → **Round 2** (multi-agent in-process) → **distributed A2A** variant — so each tool is assessed at increasing levels of complexity.

4 Approach & Method

Requirements framework. Six observability areas — reasoning traceability, multi-agent coordination, performance, quality, safety, and governance — each decomposed into concrete metrics, logs, events, and traces.

Evaluation design. A three-pillar framework (integration, signal coverage, usability & operations) applied identically to every tool, across both rounds plus the distributed variant, so capability is compared at increasing complexity.

Workloads. Five fixed query workloads per exporter, run against each system — 15 experiment result sets in total (three systems × five exporters), exported as JSON with trace screenshots.

Reproducibility. The LLM-as-judge is fixed to temperature 0; all SDK versions are pinned; and a pluggable exporter flag isolates platform-specific code so the same agent runs unchanged across all three tools.

Instrumentation. OpenTelemetry spans are emitted in both OTel GenAI and OpenInference conventions; cost is attributed per LLM call; sessions are grouped per platform; and cross-process sessions are propagated via W3C Baggage in the distributed variant.

5 Main Results

5.1 Single-agent baseline

All three tools captured the ReAct loop's reasoning, arithmetic tool calls, and token cost out of the box, with low integration effort. End-to-end latency and cost (5-query average per session) were closely matched:

TOOL	LATENCY (MEAN)	COST / SESSION
Arize Phoenix	8,536 ms	$0.0175
Langfuse	8,850 ms	$0.0177
Comet Opik	11,218 ms	$0.0182

5.2 Multi-agent in-process

The Orchestrator → Researcher → Evaluator pipeline produced richer traces: tool-call spans, inter-agent messages, LLM-as-judge scores (faithfulness, completeness, guardrail compliance), and four safety guards (loop detection, PII, token explosion, HITL escalation). In-process session grouping held across all three tools.

5.3 Distributed A2A

Researcher and Evaluator run as independent A2A JSON-RPC services; the orchestrator calls them over HTTP. Genuine cross-process span correlation required W3C context propagation (*traceparent* + *session.id* promoted into Baggage). This is where the tools diverged most.

5.4 Cross-tool comparison

TOOL	STRENGTHS	WEAKNESSES	BEST FIT
Arize Phoenix	OTel/OpenInference-native; automatic LangChain instrumentation; lowest measured latency; mature default dashboards and UI filtering options; powerful API.	Hard to debug and filter JSON-RPC calls in Agent2Agent setups; no visual representation for LangGraph states.	Any environment — aligns closely with OpenInference and backed by an active GitHub community.
Langfuse	ClickHouse-backed; strong session views and evaluation UI; mature dashboards.	Requires managing multiple resources (ClickHouse, MinIO, PostgreSQL); generates a high volume of traces per session.	Mature teams and production-grade environments.
Comet Opik	Native *thread_id* sessions; distributed-tracing support via trace-id headers.	Highest measured latency; distributed tracing only conditionally active; UI is less mature.	Local development for data scientists and ML practitioners.

5.5 What worked, what didn't

Worked: OTel-based portability let the same agent run unchanged across all three tools; per-call cost attribution; in-process session grouping.
Didn't: fully portable distributed correlation; consistent semantic conventions across tools; quality and governance measurement are still evolving and often require custom instrumentation or evaluation workflows.

6 Key Insights

- Single-agent tracing is relatively easy and well-supported across all three tools.
- Multi-agent and distributed setups create the real differentiation between platforms.
- Open standards (OpenTelemetry) materially help portability, but tool-specific attributes and conventions still matter.
- Cross-process session propagation (W3C Baggage) is critical — and not equally supported.

- Quality and governance metrics remain immature in current tooling and largely require custom instrumentation.

7 Risks, Limitations & Assumptions

- Tools differ in semantic conventions (OTel GenAI vs OpenInference), so traces are not always directly comparable.
- Several agent-specific signals (hallucination, guardrail compliance) required custom instrumentation rather than native support.
- Distributed trace correlation is not equally portable across the three tools.
- Results come from a controlled experimental setup (fixed queries, local Docker), not production traffic.
- LLM non-determinism can affect individual runs; mitigated by temperature 0 and fixed workloads, but not eliminated.

8 Recommendation

Based on the evaluation, the recommended tool selection per use case:

Default for development / debugging: Arize Phoenix

Production governance / compliance: Arize Phoenix

LangGraph-heavy workflows: Langfuse

When a hybrid approach makes sense: Arize Phoenix

9 Next Steps

- Standardise agent identifiers and span attributes across tools.
- Add native quality and safety metrics (hallucination, guardrail compliance) rather than relying on custom instrumentation.
- Expand testing to multi-turn conversations.
- Validate in a production-like environment with real traffic.
- Refine dashboards and alerting — channels, triggers, and tiers.

10 Conclusion

The project delivered a working, reproducible comparison of three open-source observability platforms across single-agent, multi-agent, and distributed architectures, backed by instrumented experiment runs and a full requirements framework. The defined objectives were met.

The practical contribution is evidence-based view of where today's open-source agent-observability tooling is strong (single-agent tracing, cost attribution, OTel portability) and where it still falls short: distributed correlation and evolving quality and governance signals implementation.

11 Appendix

A · Experiment setup

3 systems × 5 exporters × 5 query workloads = 15 instrumented run sets (JSON + screenshots). Pinned SDKs: *arize-phoenix ≥ 4.0.0, langfuse ≥ 4.12.0, opik ≥ 2.1.4, a2a-sdk 1.1.0*.

B · Glossary

ReAct reasoning-and-acting loop · **span / trace** unit / tree of instrumented work · **OTel** OpenTelemetry · **OpenInference** LLM-specific span convention · **A2A** Agent2Agent protocol · **HITL** human-in-the-loop · **W3C Baggage** cross-process context propagation · **LLM-as-judge** model-scored evaluation.

C · References

Full reference list in the technical final report.